Zephyr Project

IT-Schulungen.com
Technology & Transformation

Vom Devicetree zum Zephyr-Gerät

DTS, Binding, Treiber, Device-API und Shell

3.03

Verarbeitungskette bis zur Shell

Compile-Time-Beschreibung und Laufzeitansicht gehören zusammen

01	02	03	04	05
Devicetree	**Binding**	**Generierung**	**Treiber**	**Shell**
Board, Includes und Overlay	Schema und Property-Typen	C-Daten entstehen	struct device entsteht	device list zeigt die Instanz

Der Build übersetzt die Beschreibung. device list prüft die registrierte Instanz zur Laufzeit.

Drei Ebenen der Gerätebeschreibung

BOARD-DTS

Basis des Targets

SoC, Busse und Board-Peripherie kommen aus der Board-Beschreibung.

DTSI-INCLUDES

Gemeinsame Definitionen

SoC- und Familienbeschreibungen werden eingebunden und wiederverwendet.

APP.OVERLAY

Sicht der Anwendung

```
Knoten aktivieren
Properties überschreiben
fachliche Rollen ergänzen
```

Der finale Devicetree entsteht aus allen Ebenen. Das Overlay ändert nur die Anwendungssicht.

Begriffe am Devicetree-Knoten

Knotenname	serial beschreibt die Geräteklasse
Unit Address	Der Wert hinter @ bezeichnet die Adresse am Bus
Label	console_uart: ist eine lokale Referenz im DTS
Node Identifier	Der Build erzeugt daraus einen eindeutigen C-Bezeichner
Alias	/aliases bietet einen stabilen Zugriff auf einen Knoten

Name, Label, Identifier und Alias erfüllen verschiedene Aufgaben und sind nicht austauschbar.

compatible verbindet Knoten und Binding

APP.OVERLAY

```
gpio_emul: gpio-emulator {
    compatible = "zephyr,gpio-emul";
    gpio-controller;
    #gpio-cells = <2>;
    status = "okay";
};
```

01 compatible

Wählt das passende Binding anhand des Hersteller- und Gerätenamens.

02 Binding

Beschreibt erlaubte Properties, Datentypen und Pflichtangaben.

03 Validierung

Der Build meldet fehlende oder falsch typisierte Properties.

Das Binding macht aus freiem DTS-Text eine prüfbare Gerätebeschreibung.

Properties, Zellen und Geräte-Specifier

APP.OVERLAY

```
status_led: led {
    gpios = <&gpio_emul 0
             GPIO_ACTIVE_HIGH>;
};
```

01 Property

gpios verknüpft den Verbraucher mit einem GPIO-Controller.

02 Zellen

Das Binding des Controllers legt Anzahl und Bedeutung der Zellen fest.

03 Specifier

Controller, Pin und Flags bilden den Specifier.

Der Specifier enthält alle Informationen, die der GPIO-Treiber zur Konfiguration benötigt.

status steuert die Geräteinstanziierung

APP.OVERLAY

```
&uart0 {
    status = "okay";
};

&unused_bus {
    status = "disabled";
};
```

01 **status = "okay"**

Der Knoten nimmt am Build teil. Ein passender Treiber kann eine Device-Instanz erzeugen.

02 **status = "disabled"**

Der Knoten bleibt beschrieben, erzeugt aber keine aktive Device-Instanz.

compatible, Kconfig und Treiberinitialisierung bestimmen zusätzlich, ob das Gerät nutzbar wird.

Generierte Devicetree-Artefakte

TERMINAL

```
west build -b native_sim -d build/3_03 .
```

zephyr.dts	zusammengeführter und aufgelöster Devicetree
devicetree_generated.h	Makros, Node Identifier und Property-Werte für C
zephyr.dts.pre	Zwischenstand nach der Vorverarbeitung
edt.pickle	internes EDT-Modell für weitere Build-Schritte
include/generated/	vom Build erzeugte Header und Konstanten

Die Artefakte zeigen, welche Beschreibung der Compiler tatsächlich verwendet.

Drei Ebenen eines Zephyr-Geräts

01	`struct device`	Gemeinsamer Laufzeit-Handle und Eintrag in device list
02	`Konfigurationsdaten`	Konstante Instanzdaten aus Devicetree und Build
03	`Laufzeitdaten`	Veränderlicher Zustand und privater Treiberkontext

Die Shell inspiziert registrierte Geräte. Anwendungen nutzen die Device-API.

Vom DTS-Knoten bis zur Laufzeitinspektion

Die Verarbeitungskette in fünf Stufen erklären

01	02	03	04	05
DTS-Knoten	**Binding**	**Generierte Daten**	**Treiber**	**Shell**
beschreibt das Gerät	prüft Properties und Typen	liefern C-Konfiguration	registriert struct device	device list zeigt die Instanz

Lernziel: jede Stufe mit Quelltext, Build-Artefakt oder Shell-Ausgabe verbinden.